

The Complete Linked List Roadmap (Beginner → Advanced, Java)

70 curated problems across **LeetCode (LC)**, **GeeksforGeeks (GfG)**, and **HackerRank (HR)**. A quick note before the tables: **LeetCode is the only one of the three platforms that uses numeric problem IDs**. GfG and HackerRank identify problems by title only (no official number) — the Problem # column shows "—" for those, and the exact title is the identifier you'd search for on the platform. Titles below are matched against each platform's current practice-page wording; GfG occasionally tweaks exact phrasing, so treat the title as "search for this," not a guaranteed byte-for-byte match.

🔥 = Must-Do for this level.

Level 0 — Fundamentals & Basic Operations

Patterns: Linked List Fundamentals, Singly LL, Doubly LL, Circular LL, Traversal, Insert/Delete

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Pr
1	HackerRank	—	Print the Elements of a Linked List	Easy	Fundamentals / Traversal	—

2	HackerRank	—	Insert a node at the head of a linked list	Easy	Insert/Delete Ops	#1
3	HackerRank	—	Insert a Node at the Tail of a Linked List	Easy	Insert/Delete Ops	#2
4	HackerRank	—	Insert a node at a specific position in a linked list	Easy	Insert/Delete Ops	#2
5	HackerRank	—	Delete a Node	Easy	Insert/Delete Ops	#2
6	HackerRank	—	Print in Reverse	Easy	Recursive Traversal	#1
7	HackerRank	—	Compare two linked lists	Easy	Traversal / Comparison	#1
8 🔥	LeetCode	707	Design Linked List	Medium	Fundamentals (Singly + build toward Doubly)	#2

9	HackerRank	—	Inserting a Node Into a Sorted Doubly Linked List	Easy	Doubly LL Insert	#8
10	HackerRank	—	Reverse a doubly linked list	Easy	Doubly LL Basic Op	#9
11	GfG	—	Reverse a Doubly Linked List	Easy	Doubly LL (extra rep)	#1
12	GfG	—	Circular List Traversal	Easy	Circular LL Fundamentals	#1
13 	LeetCode	203	Remove Linked List Elements	Easy	Delete-by-Value	#5

Must-Do & key pattern: #8 (Design Linked List) and #13 (Remove Elements) — together they force you to handle head-pointer edge cases explicitly, which is the single most common source of bugs in every level that follows. Everything else here is repetition across three platforms so the *mechanics* (not just the concept) become automatic.

Level 1 — Easy Pointer Problems

Patterns: Reverse Linked List, Middle of Linked List, Nth Node from End

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequ
-------	----------	-----------	--------------	------------	---------	---------

14 	LeetCode	206	Reverse Linked List	Easy	Reversal	Level 0
15	GfG	—	Reverse a Linked List	Easy	Reversal (extra rep, do iterative + recursive)	#14
16 	LeetCode	876	Middle of the Linked List	Easy	Fast & Slow	Level 0
17	GfG	—	Nth node from End of Linked List	Easy	Two Pointers / Gap-of-N	Level 0
18 	LeetCode	19	Remove Nth Node From End of List	Medium	Two Pointers + Dummy Node	#17
19	LeetCode	83	Remove Duplicates from Sorted List	Easy	Same-Direction Traversal	Level 0
20	GfG	—	Remove Duplicates from Sorted Linked List	Easy	Same-Direction Traversal (rep)	#19
21	LeetCode	237	Delete Node in a Linked List	Easy	Trick: copy next node's value, skip it	Level 0

Must-Do & key pattern: #14 (Reverse) and #16 (Middle) are the two most-reused primitives in the entire roadmap — nearly every "advanced" problem later calls one or both as a subroutine. #18 introduces the **dummy node trick** (a sentinel node before `head` so you never special-case "deleting the head").

Level 2 — Fast & Slow Pointers + Cycle Detection

Patterns: Fast & Slow Pointers, Cycle Detection & Removal

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prereq
22 🔥	LeetCode	141	Linked List Cycle	Easy	Fast & Slow / Cycle Detection	Level 1
23	GfG	—	Detect Loop in linked list	Easy	Cycle Detection (rep)	#22
24 🔥	LeetCode	142	Linked List Cycle II	Medium	Find Cycle Entry Point	#22
25	GfG	—	Find the Length of the Loop	Medium	Cycle length via Floyd's meeting point	#24

26	HackerRank	—	Cycle Detection	Easy	Cycle Detection (3rd platform rep)	#22
27 	GfG	—	Detect and Remove Loop in Linked List	Medium	Cycle Removal	#24

Must-Do & key pattern: #22 is *the* archetype — different speeds guarantee the fast pointer laps the slow one inside any cycle, so they must eventually collide. #24/#27 extend this to *finding* and *removing* the cycle: reset one pointer to `head` after the first meeting, advance both at speed 1 — they meet exactly at the cycle's start (a consequence of the math behind Floyd's algorithm).

Level 3 — Reversal Patterns (Deeper)

Patterns: Reverse Linked List, Reverse in K Groups

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequ
28 	LeetCode	92	Reverse Linked List II	Medium	Partial (bounded-range) Reversal	Level 1
29 	LeetCode	25	Reverse Nodes in k-Group	Hard	Reverse in K Groups	#28

30	GfG	—	Reverse a Linked List in groups of given size	Hard	Reverse in K Groups (rep)	#29
31	LeetCode	24	Swap Nodes in Pairs	Medium	Reversal special case (k=2)	#28
32	LeetCode	1721	Swapping Nodes in a Linked List	Medium	Two Pointers (kth-from-each-end swap)	Level 1

Must-Do & key pattern: #28 forces you to reverse only a *sub-range*, which means tracking the node just before the range (another dummy-node use case) and reconnecting both ends afterward. #29 chains that skill k nodes at a time — it's usually the single hardest "core" linked list problem in interviews, so budget real time here.

Level 4 — Merge / Intersection / Duplicate / Palindrome

Patterns: Merge Two Sorted Lists, Intersection of Linked Lists, Duplicate Removal, Palindrome Linked List

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Pr
33 🔥	LeetCode	21	Merge Two Sorted Lists	Easy	Merge	Le

34	HackerRank	—	Merge two sorted linked lists	Easy	Merge (rep)	#3
35 	LeetCode	160	Intersection of Two Linked Lists	Easy	Intersection	Le
36	GfG	—	Intersection Point in Y Shaped Linked Lists	Easy	Intersection (rep)	#3
37	HackerRank	—	Find Merge Point of Two Lists	Easy	Intersection (3rd platform rep)	#3
38	LeetCode	82	Remove Duplicates from Sorted List II	Medium	Duplicate Removal (delete <i>all</i> dupes)	Le
39	HackerRank	—	Delete duplicate-value nodes from a sorted linked list	Easy	Duplicate Removal (rep)	#3
40 	LeetCode	234	Palindrome Linked List	Easy	Fast/Slow + Reverse + Compare	Le Le
41	GfG	—	Palindrome Linked List	Easy	rep	#4

Must-Do & key pattern: #33 (one pointer per list, always advance the smaller) is the seed for Merge K Sorted Lists later. #35's "switch heads to equalize remaining distance" trick is one of the most elegant tricks in

the whole roadmap — walk through *why* it works (both pointers travel `lenA + lenB` total steps, so they're guaranteed to align). #40 chains three earlier skills together: find the middle, reverse the second half, compare.

Level 5 — Medium Interview Problems (Partition, Rearrange, Add Numbers)

Patterns: Partitioning/Rearrangement, Add Numbers Using Linked Lists

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	
42 🔥	LeetCode	86	Partition List	Medium	Partitioning (two sub-lists, stitch together)	L
43	LeetCode	328	Odd Even Linked List	Medium	Rearrangement (interleave by index parity)	L
44	GfG	—	Segregate Even and Odd Nodes in a Linked List	Medium	Rearrangement (rep)	#
45	GfG	—	Sort a List of 0s, 1s and 2s	Medium	Partitioning (Dutch flag on a LL)	#
46 🔥	LeetCode	143	Reorder List	Medium	Rearrangement (mid + reverse + merge)	L L L
47 🔥	LeetCode	2	Add Two Numbers	Medium	Add Numbers Using LL	L

48	LeetCode	445	Add Two Numbers II	Medium	Add Numbers (no list-reversal allowed variant)	#
49	GfG	—	Add two numbers represented by Linked Lists	Medium	rep	#

Must-Do & key pattern: #42's "build two separate lists (less-than and greater-or-equal), then join them" is the general recipe for most linked-list partitioning problems. #46 (Reorder List) is a genuine capstone — it's literally #16 + #28 + #33 chained together, which is exactly why it's placed here instead of earlier. #47 teaches carrying a "carry" digit forward, node by node, the linked-list analogue of grade-school addition.

Level 6 — Advanced Manipulation (Rotate, Sort, Recursive)

Patterns: Rotate Linked List, Recursive Linked List Problems

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prereq
50 	LeetCode	61	Rotate List	Medium	Rotate LL (find new tail, re-link, break)	Level 1
51	GfG	—	Rotate a Linked List	Medium	rep	#50

52 	LeetCode	148	Sort List	Medium	Merge Sort on a Linked List	Level 2 Level 4
53	GfG	—	Merge Sort for Linked List	Medium	rep	#52
54	LeetCode	109	Convert Sorted List to Binary Search Tree	Medium	Recursive LL + Tree Construction	Level 2
55	LeetCode	1290	Convert Binary Number in a Linked List to Integer	Easy	Simple Recursive/ Iterative Traversal	Level 0

Must-Do & key pattern: #50 requires computing the list length first (a full pass), then a second pass to find the new tail — a good lesson in when "just use fast/slow" *doesn't* directly apply. #52 is the first problem where recursion and Two Pointers combine at scale: find the middle (splitting point), recursively sort each half, merge — $O(n \log n)$ with only $O(\log n)$ extra space (the recursion stack), a genuinely important algorithm to know cold.

Level 7 — Random Pointer / Multilevel / Merge K / LRU Cache

Patterns: Random Pointer/Clone, Multilevel/Flatten, Merge K Sorted Lists, Linked List + HashMap

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite
56 	LeetCode	138	Copy List with Random Pointer	Medium	Random Pointer / Deep Clone	Level 0, HashMap
57	GfG	—	Clone a linked list with next and random pointer	Medium	rep	#56
58 	LeetCode	430	Flatten a Multilevel Doubly Linked List	Medium	Multilevel Flatten	Level 0, Level 3
59	GfG	—	Flattening a Linked List	Medium	rep (GfG's version: merge sorted child lists)	#58
60 	LeetCode	23	Merge k Sorted Lists	Hard	Merge K Sorted Lists	Level 4
61	GfG	—	Merge K Sorted Linked Lists	Hard	rep	#60
62 	LeetCode	146	LRU Cache	Medium	Linked List + HashMap (O(1) get/put)	Level 0, HashMap

Must-Do & key pattern: #56 has two standard solutions worth knowing both: HashMap of {original node: cloned node} (O(n) space), or the O(1)-space "interweave cloned nodes into the original list" trick — know both, interviewers often ask for the second as a follow-up. #60 generalizes #33 (merge two lists) using a min-heap (or divide-and-conquer pairwise merging) to merge k lists in O(n log k). #146 (LRU Cache) is a design-pattern capstone: a **doubly linked list + HashMap** gives O(1) get/put/evict — this exact combination (Level 24's "Linked List + HashMap" pattern) shows up constantly in system-design-adjacent interview questions.

Level 8 — Complex / Hard / Advanced

Patterns: Advanced & Hard Linked List Problems

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	
63	LeetCode	2074	Reverse Nodes in Even Length Groups	Medium	Advanced Reversal (variable group sizes)	
64	LeetCode	725	Split Linked List in Parts	Medium	Advanced Partitioning	
65	LeetCode	817	Linked List Components	Medium	Linked List + HashSet	
66 	LeetCode	2130	Maximum Twin Sum of a Linked List	Medium	Fast/Slow + Reverse + Two Pointers combined	
67	LeetCode	1019	Next Greater Node In Linked List	Medium	Linked List + Monotonic Stack	

68 	GfG	—	Rearrange a given linked list in-place	Hard	Advanced Rearrangement (interleave first/second half)
---	-----	---	--	------	---

Must-Do & key pattern: #66 and #68 are essentially the same idea (find the middle → reverse the second half → walk both halves together) applied to different final goals (a sum vs. an interleave) — recognizing that a "new-looking" hard problem is actually a remix of Level 2+3+4 skills is the whole point of this final level. #67 is the one problem here that isn't really about pointer *movement* — it's a reminder that "linked list" in a prompt doesn't always mean the answer is a pointer trick; sometimes it's a different data structure (monotonic stack) applied to a linked traversal.

Two Pointers, Dummy Node, HashMap & Friends — Cheat Sheet

When to use Fast & Slow pointers

- Recognize: "find the middle," "detect a cycle," "find the Nth from the end" (a slight variant — see Dummy Node below for the *exact* Nth-from-end technique).
- Core idea: `slow` moves 1 step, `fast` moves 2; they meet inside a cycle, or `fast` reaches the end first when there isn't one.

```
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
// slow is now at the middle (or the meeting point, if yo
```

When to use a Dummy Node

- Recognize: the operation might need to modify or remove the **head** itself (delete Nth node, partition, merge, remove duplicates) — a dummy node before `head` means you never need a special case for "what if we're deleting/replacing the very first node."

```
ListNode dummy = new ListNode(0, head);
// ... operate using dummy.next as the "real" head ...
return dummy.next;
```

When to use Two Pointers (general, not just fast/slow)

- Recognize: "Nth from the end," "intersection of two lists," any problem where you need to align/compare two positions without random access (arrays can index; lists can't).
- Core idea (Nth from end): advance one pointer `n` steps ahead first, then move both together — the gap keeps the trailing pointer exactly `n` from the end.

```
ListNode fast = dummy, slow = dummy;
for (int i = 0; i < n; i++) fast = fast.next;
while (fast.next != null) { fast = fast.next; slow = slow.next; }
slow.next = slow.next.next; // remove nth from end
```

When to use Recursion

- Recognize: reversal, palindrome checks, or any problem naturally defined in terms of "solve the rest of the list, then do something with the current node" — recursion mirrors a linked list's self-referential structure directly.
- Trade-off: recursion is often cleaner but costs $O(n)$ stack space, vs. $O(1)$ for an iterative pointer-based solution — know both for problems like Reverse Linked List.

```
ListNode reverseRecursive(ListNode head) {
    if (head == null || head.next == null) return head;
    ListNode newHead = reverseRecursive(head.next);
    head.next.next = head;
    head.next = null;
    return newHead;
}
```

```
}
```

When to use HashMap

- Recognize: "clone with random pointers" (map original→copy), "detect a cycle" (map node→visited, $O(n)$ space alternative to Fast & Slow), "LRU Cache" (map key→node for $O(1)$ lookup).

```
Map<Node, Node> map = new HashMap<>();  
// first pass: map.put(original, new Node(original.val))  
// second pass: map.get(original).next = map.get(original
```

When to use a Stack

- Recognize: "reverse in a specific way without breaking links," "check palindrome without extra pointer tricks," "next greater node" (monotonic stack).
- Core idea: push nodes/values as you traverse; a stack naturally reverses order (LIFO), which is handy when you'd otherwise need multiple passes.

```
Deque<Integer> stack = new ArrayDeque<>();  
for (ListNode n = head; n != null; n = n.next) stack.push  
// now popping gives you the list in reverse order
```

When to use Linked List + HashMap

- Recognize: **$O(1)$ get/put with eviction** requirements — the classic signal is "design a cache," "design a data structure with $O(1)$ X and $O(1)$ Y."
- Core idea: a doubly linked list gives $O(1)$ removal/insertion at any known node; a HashMap gives $O(1)$ lookup of *which* node to remove/move — combined, you get $O(1)$ for everything.

```
// LRU Cache skeleton  
class LRUCache {  
    Map<Integer, Node> map = new HashMap<>();  
    Node head = new Node(0, 0), tail = new Node(0, 0); //  
    // head <-> ... most recently used ... <-> tail
```



```
// on get/put: move node to front; on capacity overfl  
}
```

When to use Linked List + Sorting / Heap

- Recognize: "merge k sorted lists," "sort a linked list," anything requiring a global ordering across multiple lists or an unsorted list.
- Core idea: a min-heap of "current head of each list" lets you always extract the globally smallest next node in $O(\log k)$; for sorting a single list, merge sort (find middle, recurse, merge) is the standard $O(n \log n)$ approach since random access (needed for quicksort-style partitioning) isn't available.

```
PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -  
for (ListNode l : lists) if (l != null) pq.offer(l);  
ListNode dummy = new ListNode(0), cur = dummy;  
while (!pq.isEmpty()) {  
    ListNode node = pq.poll();  
    cur.next = node; cur = cur.next;  
    if (node.next != null) pq.offer(node.next);  
}  
return dummy.next;
```

Top 20 Must-Do Problems

1. LC 206 — Reverse Linked List
2. LC 876 — Middle of the Linked List
3. LC 19 — Remove Nth Node From End of List
4. LC 141 — Linked List Cycle
5. LC 142 — Linked List Cycle II
6. LC 92 — Reverse Linked List II
7. LC 25 — Reverse Nodes in k-Group
8. LC 21 — Merge Two Sorted Lists
9. LC 160 — Intersection of Two Linked Lists
10. LC 234 — Palindrome Linked List
11. LC 86 — Partition List

- 12. LC 143 — Reorder List
- 13. LC 2 — Add Two Numbers
- 14. LC 61 — Rotate List
- 15. LC 148 — Sort List
- 16. LC 138 — Copy List with Random Pointer
- 17. LC 430 — Flatten a Multilevel Doubly Linked List
- 18. LC 23 — Merge k Sorted Lists
- 19. LC 146 — LRU Cache
- 20. LC 707 — Design Linked List

Top 30 Interview Problems

(The 20 above, plus:) 21. LC 203 — Remove Linked List Elements 22. LC 83 — Remove Duplicates from Sorted List 23. LC 82 — Remove Duplicates from Sorted List II 24. LC 24 — Swap Nodes in Pairs 25. LC 328 — Odd Even Linked List 26. LC 445 — Add Two Numbers II 27. GfG — Detect and Remove Loop in Linked List 28. LC 109 — Convert Sorted List to Binary Search Tree 29. LC 2130 — Maximum Twin Sum of a Linked List 30. GfG — Rearrange a given linked list in-place

Beginner → Advanced Complete Sequence

Follow the Master Roadmap tables above, level by level, in Order 1 → 68. Within a level, problems are already sequenced so each builds on the one before it.

Final Summary

Total curated problems: 68 unique (across 3 platforms; some are intentional cross-platform reps of the same technique)

Platform	Count
----------	-------

LeetCode	39
GeeksforGeeks	19
HackerRank	10

Difficulty	Count (approx.)
Easy	27
Medium	34
Hard	7

Level / Pattern Group	# of Problems
Level 0 — Fundamentals & Basic Ops	13
Level 1 — Easy Pointer Problems	8
Level 2 — Fast & Slow / Cycle Detection	6
Level 3 — Reversal Patterns	5
Level 4 — Merge/Intersection/Duplicate/Palindrome	9
Level 5 — Partition/Rearrange/Add Numbers	8
Level 6 — Rotate/Sort/Recursive	6
Level 7 — Random Pointer/Flatten/Merge K/LRU	7
Level 8 — Advanced/Hard	6

(🔥 *Must-Do problems are marked inline within each level's table — 26 total across all levels.*)

Study Plans

7-Day Plan (most important problems only): Day 1: 707, 203, 206, 876 · Day 2: 19, 83, 237, 141 · Day 3: 142, 92, 25 · Day 4: 21, 160, 234 ·

Day 5: 86, 143, 2 · Day 6: 61, 148 · Day 7: 138, 23, 146

14-Day Plan (essential interview problems): Days 1–7: the 7-day plan above. Day 8: 24, 328, 445 · Day 9: 82, GfG Detect and Remove Loop · Day 10: 430, GfG Clone with Random Pointer · Day 11: 109, 1290 · Day 12: 2130, 1019 · Day 13: GfG Rearrange in-place, 725 · Day 14: full review — re-solve #206, #25, #148, #23, #146 from scratch, timed.

30-Day Plan (strong preparation): Days 1–14: the 14-day plan above. Days 15–20: all remaining Level 0–2 cross-platform reps (HackerRank + GfG duplicates) for speed and fluency. Days 21–25: all remaining Level 4–6 problems (34, 37, 39, 41, 44, 45, 49, 51, 53, 55). Days 26–29: Level 8 remaining problems (2074, 817). Day 30: mixed timed review — 5 random Must-Do + 2 random Level 8 problems, no hints.

60-Day Plan: not requested, but if wanted — repeat the 30-day plan's problem set as Days 1–30, then use Days 31–60 for the standard spaced-revision cycle described below.

Revision Strategy (same discipline as the other roadmaps in this series)

Stage	When	What to do
First attempt	Day 0	Solve fresh; identify which pattern (Fast/Slow, Dummy Node, Recursion, etc.) applies before coding.
Review 1	Day 1	Re-read your solution; explain the pointer invariant out loud.
Review 2	Day 3	Re-solve from scratch, no notes.
Review 3	Day 7	Pattern-recognition only — name the pattern, no coding.
Review 4	Day 14	Re-solve all 🔥 Must-Do problems from scratch, timed.

Final review	Day 30	Mixed set: 5 random Must-Do + 2 random Level 8 problems, no hints.
--------------	--------	--

Accuracy note: LeetCode numbers/titles were verified against LeetCode's own problem pages. GfG and HackerRank problem titles were cross-checked against current practice-page listings and multiple independent problem-list sources at the time of writing; since neither platform assigns numeric IDs, treat the exact wording as "close enough to search by" rather than a database key.